

06 — The Prototype Library

The shipped database (#0 through #54, plus #90/#91) includes a library of base prototypes you build the world from: room types, the five exit kinds, containers, furniture, wearables, and consumables. This chapter is the reference for those prototypes — the properties they define and the `<verb>_` hooks they respond to.

These are **engine prototypes**, not game content: they model space, movement, containment, clothing layers, and consumption generically. A specific game adds its own subclasses and content on top, documented separately.

Property names below are the load-bearing ones used by the engine's own verbs. For the authoritative, current list on any prototype, use `+props/all #N` in-game (and `+verbs/all #N` for verbs). See Building Worlds.

- [Rooms and the compass](#)
- [Exits](#)
- [Containers and spatial storage](#)
- [Furniture](#)
- [Wearables](#)
- [Consumables](#)

Rooms and the compass

Rooms descend from **#15 BaseRoom**, with two concrete types: **#16 OCRoom** (out-of-character) and **#17 ICRoom** (in-character). All player commands live on these two — see where verbs live.

Room properties

Property	Meaning
<code>directions</code>	Per-direction exit slots for the 12-direction compass.
<code>dexits</code>	Virtual exit data — lightweight direction links stored as room data rather than as separate exit objects.
<code>exits</code>	The object-backed exits located in this room.
<code>obvexits</code>	The exits shown in the room's "obvious exits" line.
<code>plist</code>	Players currently in the room.

The 12-direction compass

Directions are defined in `moo/globals.py`. `DNAMES` holds the canonical token for each of the 12 direction slots, in index order 0–11. Note that these tokens are not uniformly short or long: the first four are spelled out, the rest are abbreviated.

Idx	DNAMES token	Also accepted	Reverse (RETEXIT)
0	north	n	south
1	south	s	north
2	east	e	west
3	west	w	east
4	ne	northeast	sw
5	nw	northwest	se
6	se	southeast	nw
7	sw	southwest	ne
8	u	up	d
9	d	down	u
10	o	out	—
11	in	—	—

Players may type either form (`n` or `north` , `u` or `up`); `DALIASES` pairs them and `DIRECTIONS` is the flat list of every accepted token. The `RETEXIT` table maps each direction to its opposite and is what `@dig / @open` use to auto-create the **return exit** — it has no entry for `o` or `in` , so passages through those are created one-way unless you open both ends.

Room verbs

`look` / `look_here` (render the room), `go` and the directional verbs (movement), `match_exit` (resolve a direction or keyword to an exit), and `gmove` (the generic move worker exits call back into).

Exits

All exits descend from **#20 BaseExit**. The five concrete kinds and their builder commands (full details in Building Worlds):

#	Prototype	Built with	Traversed by
#21	DirectionalExit	@open / @vopen	a compass direction
#22	GoExit	@gopen	go <name>
#23	ClosableGoExit	@dopen	go <name> (must be open)
#24	ClimbableExit	@copen	climb <name>
#25	JumpableExit	@jopen	jump <name>

Exit properties

Property	Meaning
source	The room the exit leads from.
destination	The room the exit leads to.
success / osuccess	Message to the mover / to onlookers on a successful traversal.
drop / odrop	Message to the mover on arrival / to onlookers in the destination.
mode / omode	First- / third-person movement verb ("walk" / "walks").
closed	Whether a closable exit is currently closed.
locked / latched	Lock / latch state on closable exits.

Exit message tokens

Exit messages substitute these tokens (defaults in `globals.py` as `ESUCC` / `E0SUCC` / `E0DROP`):

Token	Renders
%MODE	first-person movement verb (from <code>mode</code>)
%OMODE	third-person movement verb (from <code>omode</code>)
%S	the mover's styled display name
%1 / %dir	the direction name

```

success: "You %MODE %1."      → "You walk north."
osuccess: "%S %OMODE %1."    → "Alice walks north."
odrop:   "%S %OMODE in from the %1." → "Alice walks in from the south."

```

These are distinct from the emit tokens used by `msg / msg_room` (see Writing Verbs); exit messages use the `%MODE` / `%1` set above.

Containers and spatial storage

Spatial storage is split across two prototypes:

- **#11 (object)** defines the four **spatial relations** — `in`, `on`, `under`, `behind` — that any object supports, via the `<rel>_put` / `<rel>_get` / `<rel>_look` verb family. So you can `put vase on statue` or `look under rug` even when the target isn't a "container."
- **#26 (BaseContainer)** adds an enclosed interior with capacity limits and open/closed state. Concrete types: **#27 Chest**, **#28 Cup**, **#29 Box**.

Spatial prepositions

<code>put book in chest</code>	→ <code>in_contents</code>	<code>look in chest</code>	→ shows <code>in_contents</code>
<code>put vase on chest</code>	→ <code>on_contents</code>	<code>look on chest</code>	→ shows <code>on_contents</code>
<code>put key under chest</code>	→ <code>under_contents</code>	<code>look under chest</code>	→ shows <code>under_contents</code>
<code>put note behind chest</code>	→ <code>behind_contents</code>	<code>look behind chest</code>	→ shows <code>behind_contents</code>
<code>get book from chest</code>	→ removes from <code>in_contents</code>		

Container properties

Property	Meaning
<code>open</code>	Whether the container is currently open (gates <code>in_*</code> access).
<code>in_contents</code>	Objects inside. (<code>on_</code> / <code>under_</code> / <code>behind_contents</code> exist on #11 for the other relations.)
<code>max_vol</code>	Maximum total volume the interior holds.
<code>max_items_in</code>	Maximum item count inside.
<code>max_weight_in</code>	Maximum total weight inside.
<code>pass_locks</code>	Lock-bypass configuration for the interior.

Container verbs

`in_put` / `in_get` / `in_look` (the interior operations, which enforce open state and the capacity limits), and `look_` (how the container renders when examined). The `enter_func` / `exit_func` hooks (the `after_enter` / `after_leave` aliases — see Engine Systems) run when something enters or leaves a container you can step into.

Furniture

Furniture descends from **#30 BaseFurniture**: things players sit on, lie on, or gather at. Concrete types include **#31 Chair**, **#32 Bed**, **#33 Table**, **#34 Bench**.

Furniture properties

Property	Meaning
<code>seats</code>	How many players the piece holds.
<code>sitters</code>	Players currently using it.
<code>sit_prep</code>	The preposition used — <code>on</code> (chair/bench), <code>at</code> (table), <code>in</code> (e.g. a tub).

The position system

A character's `position` property is an integer:

Value	Position
0	standing
6	sitting
7–9	lying (varying poses)

Sitting or lying on furniture sets the value and adds the character to the piece's `sitters`; `stand` clears it.

Furniture verbs

`sit_`, `lay_` (the per-object hooks the room's `sit` / `lay` verbs dispatch to, which check capacity and `sit_prep`), `look_` (renders the piece and who's on it), and `tt` — **table talk**, which speaks only to others at the same piece of furniture.

Wearables

Wearables descend from **#35 BaseWearable** (itself a child of #12 item). The shipped clothing prototypes are **#36 Shirt**, **#37 Pants**, **#38 Shoes**, **#39 Hat**, and **#40–#49 BlankClothing** (blank templates to clone and configure).

Body slots and the layer system

A wearable's `wear_pos` is a list of `[position, size]` pairs naming the body slots it occupies and how it layers there. Layering is **size-based**:

- **Size 0 — accessories.** Don't participate in layering; multiple can share a slot (rings, bracelets).
- **Size 1+ — layers.** A larger size goes over a smaller one in the same slot; you can't put a smaller garment over a larger one. Removing an inner layer is blocked while something larger covers it.

Bilateral (left/right) slots are encoded by position values over 100: the base slot is `pos % 100` (the **right** side) and the **left** is `base - 1`, so a single `wear_pos` entry can describe a left-or-right item.

Configure all of this in-game with `@wearpos` (a game/staff command — see its note in Building Worlds).

Wearable properties

Property	Meaning
<code>wear_pos</code>	List of <code>[position, size]</code> pairs — where it's worn and how it layers.
<code>worn</code>	Whether it's currently worn.
<code>size</code>	The layering size of the garment.
<code>visible</code>	Whether it's visible when worn.
<code>layer_flex</code>	Per-slot flags allowing flexible layering order.

(A character tracks what it has on via its own `wearing` property.)

Wearable verbs

`wear_` and `remove_` (the per-object hooks the room's `wear / remove` dispatch to — they check slot availability, layer order, and what's on top), plus `items_on_top` / `items_under` (the helpers that report layering conflicts and so gate removal).

Consumables

Two prototypes, both children of #10: **#90 BaseEdible** (food) and **#91 BaseDrinkable** (beverages). Both support multiple uses, effects on consumption, and custom messages.

Properties

Property	Meaning
<code>uses</code>	Bites / sips remaining.
<code>prepared</code>	Whether the item is ready to consume.
<code>effects</code>	Effect(s) applied to the consumer on each use (via the effects system).
<code>eemits</code>	Messages emitted to the room on each use.
<code>rtdice</code>	Dice parameters for randomized effect application.
<code>finish</code> / <code>ofinish</code>	Message to the consumer / to onlookers on the final use.
<code>liquid</code>	Marker on #91 identifying it as a beverage (used by cup containers).

Verbs

`eat_` (on #90), `drink_` and `sip` (on #91): each decrements `uses`, applies `effects`, shows the `eemits`, and on the last use shows `finish` / `ofinish`. They are the per-object hooks the room's `eat` / `drink` / `sip` verbs dispatch to.

```
@make #90
@name <obj> = loaf of bread
@adprop <obj>.uses = 5
@adprop <obj>.prepared = True
@adprop <obj>.finish = You eat the last of the bread.
@adprop <obj>.ofinish = %S eats the last of the bread.
```

(Property values are set with `@adprop` — there is no separate `@set`; on a fresh instance these create local overrides of the inherited defaults. `@adprop` evaluates the right-hand side as a Python expression where it can, so `5` and `True` become an int and a bool, while unparseable text is stored as a string.)

[Back to the index.](#)